PRD: dannyockilson.com

Version

v1.1 — Draft

Changelog

- **v1.1**: Folded in review comments. Added Experiments/Toys content type. Replaced TypeScript canned response config with MDX-based content model. Updated grid frame to L-shape. Added dark mode (inverted colours). Added dynamic prompt text for return visitors. Added suspicious-route Easter eggs. Clarified template hierarchy. Added AI-agent-friendly content as a success metric. Resolved open questions (prompt wording, social links, home page content).

Vision

A personal website built around two distinctive interactions: a Snellen-chart-inspired typographic grid that serves as both loading state and site identity, and a conversational search prompt that replaces traditional navigation. The name "Danny Ockilson" rendered at scale *is* the brand — no logo, no hero image. The site's personality comes through in the question it asks you — and that question changes depending on who you are and how many times you've visited.

Goals

1. **Identity through interaction** — the intro animation and the search prompt together form the entire first impression. No template energy. No generic portfolio site.
2. **Animation as functional UI** — the name grid plays on first visit as an identity moment, and on subsequent visits only when content needs async loading time. It masks latency while reinforcing brand. Return visits may show different animation treatments.
3. **Navigation through intent** — visitors declare what they're looking for rather than scanning a menu. This filters the site through the visitor's purpose and creates space for personality (Easter eggs, dynamic prompts, suspicious-activity responses).
4. **Low-friction publishing** — MDX in a git repo. Push to deploy. No CMS. All content — including Easter eggs and interactive toys — is authored as MDX.

5. **Mobile-first** — the grid layout was designed for portrait viewports. Desktop/landscape is the adaptation.
6. **AI-agent-friendly** — all content is served as well-structured MDX with semantic headings, making it consumable by AI agents and LLMs.

Non-Goals

- CMS or visual editor.
- LLM-powered search routing (future enhancement, not v1).
- A "complete" portfolio site on day one — ship the grid + search + writing first, iterate.

Deferred Goals

- **Analytics/tracking** — not in v1, but would be nice to have at least basic visit tracking. To be evaluated after launch. Should not require a server or third-party scripts if possible (consider privacy-respecting options like Plausible, Fathom, or Cloudflare Web Analytics).
- **Social features** — comments, likes, etc. Not planned but not ruled out forever.

Aesthetic Direction

Snellen eye chart as the primary reference. This means:

- Black/near-black text on white/cream background. High contrast.
- Letters are the visual architecture. No visible grid lines, no card borders, no coloured blocks.
- Size hierarchy creates structure — large letters dominate, smaller text is subordinate. The grid's spatial relationships are communicated through type size and spacing alone.
- Monospace or geometric/optotype-inspired typeface.
- Potential to layer in deeper Snellen-specific references (red/green duochrome accents, the classic chart tapering shape) — to be explored in design prototyping. Start typographic, add references if they feel right.

Dark Mode

Dark mode inverts the palette: white/cream text on black/near-black background. The duochrome bars invert accordingly. This is a natural fit for the Snellen aesthetic — eye charts are tested in dark rooms. Implementation should use `prefers-color-scheme` media query with a manual toggle available.

Visual System: Duochrome Bars

The Snellen red/green duochrome test provides a functional visual element beyond just aesthetic reference. Horizontal coloured bars (green and red) appear throughout the UI as:

- **Structural dividers** between content sections on pages
- **Result type indicators** in search results (green = top/primary match, red = secondary/additional results)
- **Framing elements** around the search input on the home page (green bar above, red bar below — or vice versa, to be explored)

These bars replace traditional horizontal rules, card borders, or section separators. They're the only colour in an otherwise black-and-white palette, which makes them visually powerful without being decorative.

Content Architecture

Content Types and Templates

The site has three content types, each with a different template relationship:

Content Type	Template	Frontmatter	Source
Writing	Astro layout template with status display	Full frontmatter (title, date, tags, description, status, draft)	MDX in `/content/writing/`
Projects	Astro layout template with references/links	Full frontmatter (title, description, tags, url, repo, featured, date)	MDX in `/content/projects/`
Experiments	No template wrapper — just MDX	Minimal frontmatter (title, description, tags, date, draft)	MDX in `/content/experiments/`

Writing pages use a structured template that displays metadata (status, date, tags) and wraps content in the standard content page layout.

Projects pages use a structured template that displays references (live URL, repo link) and wraps content in the standard content page layout.

Experiments pages are pure MDX with no template chrome beyond the persistent site frame. The MDX file is the entire page. This gives maximum flexibility for interactive toys, Easter eggs, and one-off creative pages. Experiments can also be embedded within Writing pages as React islands.

Routes

Route	Content	Source
`/`	Intro animation → home page with search prompt	Astro component
`/writing`	Blog index. Reverse-chron, titles + dates + optional tags.	Content Collection query
`/writing/[slug]`	Individual post. Full MDX rendering with template.	MDX file
`/projects`	Project index.	Content Collection query
`/projects/[slug]`	Individual project page with references.	MDX file
`/experiments`	Experiments index (optional — may not need a listing page).	Content Collection query
`/experiments/[slug]`	Individual experiment. Pure MDX, no template.	MDX file

Content Model (MDX Frontmatter)

Writing:

```
title: string
date: ISO date
tags: string[] (optional)
description: string (meta/OG and search indexing)
status: string (optional – e.g. "draft", "in progress", "complete")
draft: boolean (default false)
```

Projects:

```
title: string
description: string
tags: string[]
url: string (optional, live link)
repo: string (optional, GitHub link)
featured: boolean (default false)
date: ISO date
```

Experiments:

```
title: string
description: string (meta/OG and search indexing)
tags: string[] (optional)
date: ISO date
```

```
draft: boolean (default false)
hidden: boolean (default false — if true, excluded from listings but still
accessible via URL and search)
trigger: string[] (optional — search phrases that should resolve to this
experiment)
```

The `hidden` flag is key for Easter eggs — they don't appear in any index page but can be reached via search (if the user types the right thing) or direct URL.

The `trigger` field in experiments frontmatter allows Easter eggs to declare their own search triggers directly in the MDX file, keeping everything content-driven.

The Name Grid

Structure

The name is displayed as two typographic blocks:

- **OCKILSON** — forms an L-shaped frame. The letters run vertically down the left edge and then horizontally along the bottom edge of the viewport. This L-shape creates a persistent architectural frame around the content area.
- **DANNY** — arranged in a compact grid (roughly 2 columns × 3 rows), positioned in the top-right area, vertically offset against the vertical arm of the OCKILSON L.

No visible grid lines. Letter sizing follows a Snellen-inspired hierarchy — larger letters in DANNY (it's the first name, the "big E" at the top of the chart), smaller letters in OCKILSON. Spacing and alignment imply the grid structure.

Two States

Expanded (intro/loading): Full-screen. Letters at maximum scale. The L-shape of OCKILSON and the DANNY block fill the viewport. This is the only thing on screen.

Collapsed (content view): The OCKILSON L-frame shrinks — the vertical arm becomes a narrow sidebar on the left edge, the horizontal arm becomes a narrow footer-like bar along the bottom. DANNY shrinks to a small badge in the top-right corner. The content area fills the space within the L-frame. Below the DANNY badge: configurable icon links (LinkedIn, GitHub, plus any future additions).

Landscape / Desktop Adaptation

On wider viewports, the L-frame may need to adapt. The vertical arm of the L could be wider, or the horizontal arm could extend fully across the bottom. The DANNY block has more room and doesn't need to shrink as aggressively. To be explored in design prototyping — the core mobile-first layout is the primary design target.

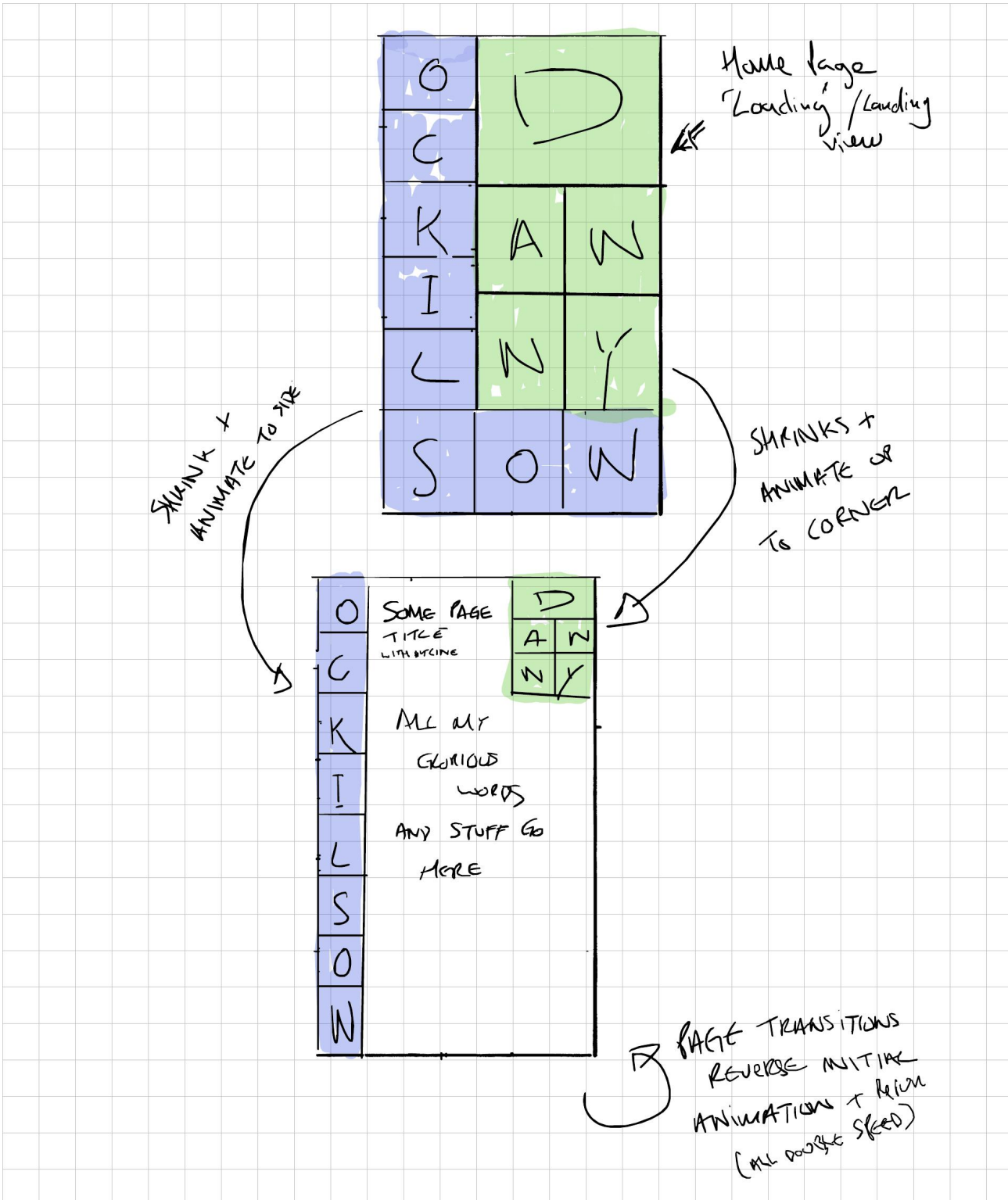

Figure 1: Original wireframe — name grid layout, expanded/collapsed states, and page transition flow

Animation Behaviour

First visit to `/`:

1. Full-screen name grid renders.
2. Grid assembles/settles (animation ~1–1.5s).
3. Grid collapses: OCKILSON to L-frame, DANNY to top-right badge.
4. Home page content (search prompt) fades into the revealed content area.
5. Icon links appear beneath DANNY badge.

Return visit (same session): No animation. Collapsed grid + home content renders immediately.

Return visit (new session, content cached/fast): No animation. Immediate render.

Return visit (new session, content loading): Animation plays as a loading state. Collapses once content is ready. Return visit animations may use different treatments/variations to keep the experience fresh.

Deep link (e.g. `/writing/some-post`): Collapsed grid renders immediately as the persistent frame. No intro animation.

`prefers-reduced-motion`: Skip all animation. Hard-cut to collapsed state.

Navigation System

Primary: Search Prompt

The home page content area displays a heading and a search input. The heading text is dynamic based on visit context:

First visit: "What do you want?"

Second visit (same session or recognised return): "Nice to see you again"

Multiple visits: "You again??"

Suspicious routes (visitor hits `/wp-admin`, `/login`, `/admin`, `/.env`, or similar common CMS/exploit routes): Redirect to home with the prompt text: "What do you think you're doing?!" — this is both a security-awareness Easter egg and a personality moment.

The exact wording for all variants is to be refined during implementation, but the system should support multiple prompt variants selected by context.

Below the heading: green duochrome bar, search input, red duochrome bar (Snellen-inspired framing).

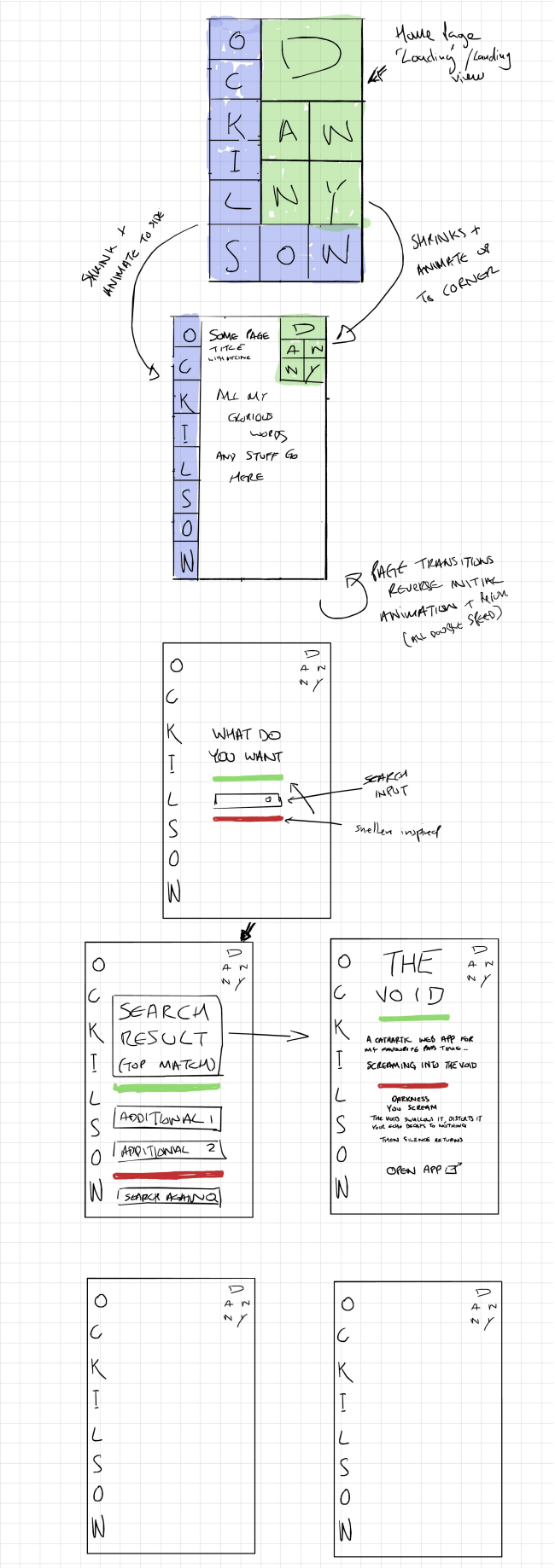

Figure 2: Updated wireframe — search prompt, results layout, content page template, and persistent frame

The search prompt is the primary navigation mechanism. Users type what they're looking for and the system resolves their intent.

Search Resolution (Priority Order)

All search resolution is content-driven. There is no separate TypeScript/JSON config file for canned responses — everything is MDX content that the search system indexes.

1. **Experiment triggers** — search query matched against the `trigger` field in experiment frontmatter. These are exact or near-exact matches that resolve to specific experiments (including Easter eggs). Because experiments declare their own triggers in frontmatter, adding a new Easter egg is just creating a new MDX file.
2. **Direct content match** — search query matched against content titles across all content types. Exact title matches auto-navigate.
3. **Common intent keywords** — a small set of navigation shortcuts baked into the search logic (not content): "blog" and "writing" → `/writing`, "projects" → `/projects`, "experiments" and "toys" → `/experiments`. These are hardcoded route aliases, not content.
4. **Fuzzy content match** — search query matched against content titles, tags, and descriptions across all content types using client-side fuzzy search (e.g. Fuse.js). Returns ranked results.
5. **No match fallback** — friendly message with suggested categories / popular content.

Search Results Layout

- **Search input** stays pinned at top of content area.
- **Top match** is displayed prominently — large title, takes up significant space. If confidence is high enough (exact trigger match, exact title match, or single fuzzy result), auto-navigates directly to the target page instead of showing this view.
- **Additional results** listed below, each separated by a coloured duochrome bar. Smaller treatment than the top match.
- **"Search again"** link/button at the bottom of results, returns focus to the search input.

Future Enhancement: LLM Routing

The content-driven search system is the v1 implementation. The architecture should support an LLM-powered intent resolver as a future additional resolution step (between triggers and fuzzy, or replacing fuzzy) without changing the UI. The LLM would operate over a collection of all MDX text content and React island metadata. This would allow natural language queries like "what do you think about CSS-in-JS" to surface relevant blog posts.

Secondary: Fallback Navigation

- After leaving the home page, a **search icon** appears alongside the social/contact icons beneath the DANNY badge. Clicking it returns to `/` with the search prompt focused.
- On the home page, while the search input is focused/active, a **"just give me a menu" link** fades in after a short delay (~2–3 seconds). This opens a floating nav overlay listing Writing, Projects, and Experiments (and any future sections). The delay is intentional — the search prompt is the intended interaction; the menu is a safety net, not a primary affordance.
- **Remember menu preference**: if a user clicks "just give me a menu", store this preference (localStorage) and show the menu link immediately on their next visit rather than with a delay. They've told you they prefer traditional nav — respect that.

Navigation on Content Pages

Once on a content page (`/writing/[slug]`, etc.), the persistent frame (OCKILSON L-frame + DANNY badge + icons + search icon) provides:

- **DANNY badge** → link back to `/`
- **Search icon** → link back to `/` with search focused
- Standard content-level navigation (next/prev post, back to index) lives within the content area, not in the persistent frame.

The DANNY Badge (Post-Collapse)

After grid collapse, the DANNY block in the top-right corner is a persistent UI element across all pages:

- Letters maintain their mini-grid arrangement (2×2-ish) at small scale
- Functions as a home link (click → `/`)
- Below it: a vertical stack of icon links:
- LinkedIn
- GitHub
- Search icon (appears after navigating away from home)
- Additional icons can be added to the config as needed
- On mobile: DANNY badge may need to be smaller than desktop to preserve content space. Icons may collapse into a single "..." or similar.

Content Page Templates

Writing Template

Per the wireframe (using Void Screamer as the reference example):

- **Title** — large, Snellen-scale typography at the top of the content area
- **Status / date / tags** — metadata line below title
- **Green duochrome bar** — divider
- **Body content** — MDX-rendered prose, images, code blocks, embedded React components (including experiments embedded as islands)
- **Red duochrome bar** — divider
- **Embedded app/interactive section** (optional) — React island embedded via MDX. For project pages this might be a live demo. "Open app ↗" links out to standalone deployed versions where applicable.

Projects Template

Same structure as Writing but with a references section:

- **Title**
- **Description/subtitle**
- **Links** — live URL, repo link (rendered from frontmatter)
- **Green duochrome bar**
- **Body content**
- **Red duochrome bar**
- **Embedded demo** (optional)

Experiments (No Template)

Experiments pages are pure MDX. The only wrapper is the persistent site frame (OCKILSON L-frame + DANNY badge). Everything else — layout, content, interactivity — is defined in the MDX file itself. This means an experiment can be:

- A full-screen interactive app
- A text page with an embedded React component
- A visual gag or joke that takes over the content area
- Literally anything that can be expressed in MDX + React

Embedded React Islands in MDX

A core feature of the site is the ability to embed interactive applications directly within content pages. This is not an edge case — it's a primary use case for both project showcases and Easter egg responses.

Examples

- **Project pages**: live demos embedded inline (e.g. Void Screamer playable directly on its project page, with an "Open app ↗" link to a standalone version)
- **Experiments**: standalone interactive experiences that are their own pages (e.g. typing a specific phrase in search loads the experiment page)
- **Blog posts**: interactive code demos, data visualisations, or illustrative tools embedded alongside prose. Experiments can be imported and embedded within writing posts as React islands.

MDX makes this native — a blog post can `import` a React component and render it inline. Astro's `client:` directives control when the JS loads (on page load, on visible, on idle, etc.).

JS Policy

Minimal framework JS by default — Astro's island architecture means pages only ship JS for interactive components. Content pages ship no JS *unless* the MDX embeds a React component (which is expected and encouraged). The site should embrace MDX's ability to embed interactive mini-apps, demos, and Easter egg components directly in posts and project pages. This is a feature, not an exception.

Suspicious Route Handling

When a visitor hits a common CMS or exploit route, the site should catch it and redirect to home with a special prompt. This serves as both a personality moment and a subtle acknowledgment.

Routes to Catch

- `/wp-admin`, `/wp-login.php`, `/wp-content`
- `/admin`, `/administrator`
- `/login`, `/signin`
- `/.env`, `/.git`
- `/phpmyadmin`
- `/xmlrpc.php`
- Any other common bot/scanner targets

Behaviour

Redirect to `/` with a query parameter (e.g. `/?suspicious=true`) that causes the prompt heading to display: **"What do you think you're doing?!"** (or similar). The search input still works normally — this is just a different greeting.

This could be implemented as a catch-all route in Astro that checks against a blocklist and redirects, or as a Cloudflare Pages redirect rule.

User Flows

Flow 1: First-Time Visitor

1. Arrives at `/`. Full-screen name grid animates in.
2. Grid collapses into L-frame. Home content reveals: heading ("What do you want?"), search input framed by duochrome bars.
3. Visitor types "blog" → auto-navigates to `/writing`.
4. Reads blog index. OCKILSON L-frame + DANNY badge + search icon visible as persistent frame.
5. Clicks a post → `/writing/some-post`. Reads it.
6. Clicks DANNY badge → returns to `/`. No animation (same session). Search prompt visible with return-visit heading.

Flow 2: Return Visitor (Same Session)

1. Navigates to `/`. No animation. Home content with search prompt renders immediately.
2. Heading shows "Nice to see you again" or "You again??" depending on visit count.
3. Types a query → results or direct navigation.

Flow 3: Return Visitor (New Session, Slow Connection)

1. Navigates to `/`. Content needs loading. Name grid animation plays as loading indicator — may use a different animation treatment than first visit.
2. Content ready → grid collapses → home content reveals.

Flow 4: Deep Link from Search Engine

1. Arrives at `/writing/some-post`. Collapsed grid frame renders immediately.
2. Reads content. Notices DANNY badge, search icon, social links in top-right.
3. Clicks search icon → navigates to `/` with search focused.

Flow 5: Frustrated User

1. Arrives at `/`. Animation plays. Search prompt appears.
2. Stares at search input. Doesn't want to type.
3. After ~2–3 seconds, "just give me a menu" link fades in.
4. Clicks it → floating nav overlay appears with Writing / Projects / Experiments.
5. Preference is stored — next visit, menu link appears immediately.
6. Nav overlay also shows dynamic content: latest posts, popular experiments, etc.

Flow 6: Easter Egg

1. Visitor types a specific phrase into search (e.g. a known Easter egg trigger from an experiment's frontmatter).
2. Single match → auto-navigates to the experiment page.
3. Experiment page renders as pure MDX — could be anything from a full interactive app to a visual joke.

Flow 7: Suspicious Route

1. Bot or curious visitor hits `/wp-admin`.
2. Redirected to `/?suspicious=true`.
3. Prompt heading shows "What do you think you're doing?!"
4. Search input works normally. Visitor can still navigate the site.

Flow 8: AI Agent

1. AI agent or crawler accesses any content page.
2. Content is served as well-structured HTML generated from MDX with semantic headings.
3. Agent can parse and understand the content structure. All text content and metadata is accessible.

Mobile Considerations

- **Grid animation:** Designed mobile-first. The L-frame (OCKILSON vertical + horizontal) works in portrait. Collapsed state: vertical arm becomes a very narrow left strip, horizontal arm becomes a narrow bottom bar. DANNY badge is small in top-right corner. Content area gets maximum space within the L.
- **DANNY badge size:** Smaller on mobile than desktop. May need to reduce the letter grid to just "D" or a minimal mark on very narrow viewports.
- **Search input:** Full-width in the content area. Standard mobile keyboard interaction.

- **"Just give me a menu" button:** Same behaviour as desktop. Floating nav overlay is touch-friendly.
- **Icon links:** May need to collapse or reduce on very narrow viewports.

Technical Constraints

- **Static site** — SSG output, no server required at runtime.
- **Minimal framework JS** — Astro island architecture. Pages only ship JS for embedded React components. Many pages *will* have interactive islands; this is by design.
- **MDX for all authored content** — writing, projects, experiments, and Easter eggs are all MDX files. Embedded React components are a first-class pattern.
- **No separate canned response config** — all searchable content lives in MDX frontmatter. The search system indexes content, not a config file.
- **Typed frontmatter** — build-time validation of content schema via Astro Content Collections + Zod.
- **Git-push deploy** — no manual build/deploy steps.
- **Dark mode** — supported via `prefers-color-scheme` + manual toggle. Implemented as palette inversion.

Success Metrics

1. The site exists and is live.
2. The animation feels polished and intentional, not gimmicky.
3. The search prompt is actually usable — people can find content through it.
4. Danny publishes at least one blog post within the first month.
5. The site doesn't look like any other developer portfolio.
6. All content is served as well-structured MDX (with semantic headings) making it consumable by AI agents.

Implementation Phases

Phase	Scope	Notes
1. Foundation	Astro setup, Content Collections (writing, projects, experiments), content layouts + templates, MDX pipeline, deploy pipeline. Static collapsed-grid L-	Gets a functional site live. Search works. Content

Phase	Scope	Notes
	frame layout (no animation). Home page with search prompt (trigger matching + fuzzy search).	renders. All three content types functional.
2. Grid Animation	Name grid component in both states (expanded + collapsed L-frame). GSAP intro animation. Session-aware animation logic (play/skip). Loading-state behaviour. `prefers-reduced-motion`. Return-visit animation variations.	The identity layer. Site goes from "functional" to "distinctive".
3. Polish	Snellen aesthetic refinement (typography, spacing, duochrome bar treatment). Dark mode support (inverted colours). Mobile refinements (DANNY badge sizing, touch targets). OG images, meta tags, favicon, RSS feed, 404 page.	The craft layer.
4. Personality	Easter egg experiments. Dynamic prompt text (visit-count-aware). Suspicious route handling. "Just give me a menu" with preference memory. Refined search fallback messaging.	The delight layer.
5. Content	Actual blog posts, project write-ups, and experiments/toys. This is the ongoing phase — the site is only as good as what's on it.	The substance layer. Never "done".

Resolved Questions (from v1.0)

1. **Prompt wording** — resolved. Dynamic based on context: "What do you want?" (first visit), "Nice to see you again" (return), "You again??" (frequent), "What do you think you're doing?!" (suspicious routes). Exact wording to be refined during implementation.
2. **Social/contact links** — LinkedIn + GitHub. Architecture supports adding more later.
3. **Home page curated content** — just the prompt. No extra content below the search input. The home page is the search prompt and nothing else.
4. **LLM search upgrade path** — future enhancement. The LLM would operate over a collection of all MDX text content and React island metadata. Architecture (MDX-based content, frontmatter triggers) supports this cleanly.

Open Questions (Remaining)

1. **Snellen depth** — start with typographic approach (size hierarchy, high contrast B&W, monospace/optotype font) and explore duochrome red/green accents and chart-shape references in prototyping. The duochrome bars are confirmed as a UI element; the question is how far to push the broader Snellen aesthetic.

2. **L-frame proportions** — the OCKILSON L-shape (vertical left arm + horizontal bottom arm) needs design exploration. How thick are the arms? How do the letter sizes distribute across the L? Does the horizontal arm feel like a footer or more like a frame edge?
3. **Experiments index** — does `/experiments` need a public listing page, or should experiments only be discoverable via search? If hidden experiments are Easter eggs, a public listing might spoil them. Could have a public listing that only shows non-hidden experiments.
4. **CV/Resume route** — may want this eventually. Not in v1 scope.
5. **Analytics solution** — deferred but desired. Evaluate privacy-respecting options post-launch.
6. **Return-visit animation variations** — how many different loading animations? How are they selected? Random? Sequential? This is a Phase 2 detail.